個人練習專案 • 實作路線圖

Multi-Agent Chatbot 後端
從零到壓測與分散式

把學習手冊第 1 章到第 13 章，串成一個完整、可放上履歷的專案。
只列步驟與檢查點，不附程式碼：邊做邊學。

版本日期：2026-10-02
搭配文件：《LLM Agent 後端壓測與分散式系統：初學者學習手冊》
預估總時數：約 6–8 週，每週 8–12 小時

0. 專案總覽

你要做的是一個「多代理人（multi-agent）客服聊天機器人」的後端：使用者送出一個問題，由一個主管 agent 判斷要找哪些專家 agent 處理，最後再經過一個審查 agent 才回覆。這種結構讓一個使用者請求觸發 3 到 4 次 LLM 呼叫，正好是壓測、async、容錯和分散式最能發揮的情境。

做完之後你會有什麼

- 一個 GitHub repo：多代理人後端、OpenAI 相容的模擬 LLM、Locust 腳本、Docker Compose 部署、Grafana 儀表板。
- 一份實驗紀錄表：每一次壓測的設定和結果，每個履歷數字都能追溯到其中一列。
- 至少 5 張對比圖：sync vs async、加入容錯前後、1 台 vs 3 台、有無快取、本機 vs 雲端。
- 一段可以在面試中完整講 10 分鐘的專案故事。

專案主題建議

建議做「個人貸款諮詢助理」，全部使用自己編的假資料（假的利率表、假的 FAQ、假的客戶）。如果擔心和 Volvo 的工作內容太接近，也可以換成「旅遊規劃助理」或「大學選課助理」，架構完全一樣。**程式碼一律自己從零寫，不參考、不複製公司的任何程式或資料。**

四個 agent 的分工

Agent	負責什麼	LLM 呼叫次數
Supervisor（主管）	讀懂問題，決定要找哪一個或哪幾個專家	1 次
Knowledge（知識）	從一份小型假 FAQ 找相關段落，整理成回答	0–1 次
Calculator（試算）	呼叫「月付金試算」工具（純 Python 函式），解釋結果	0–1 次
Reviewer（審查）	檢查最終回答是否一致、是否附上必要提醒，必要時修正	1 次

Knowledge 和 Calculator 都需要時，兩者互不依賴，可以同時執行。這會是你在第四階段練習 `gather` 的實戰場景。

最終架構

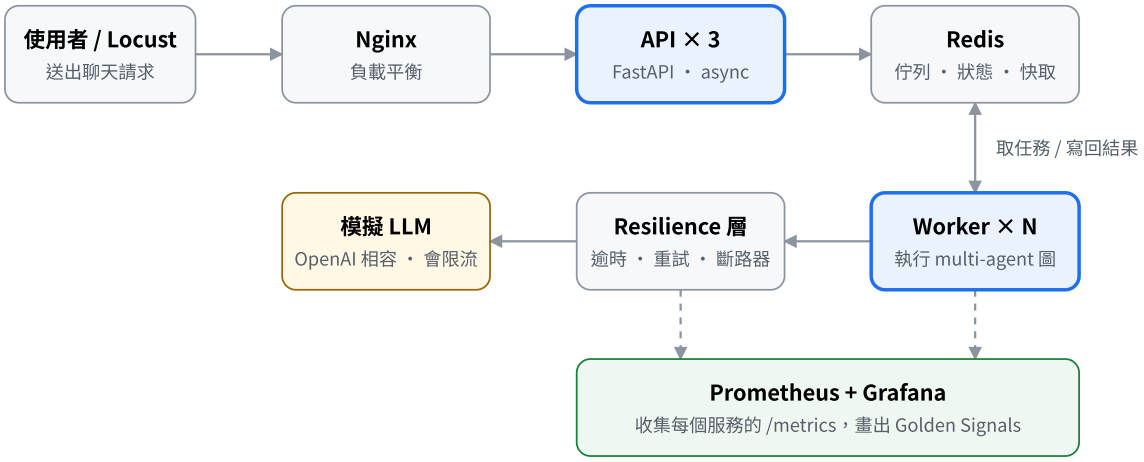


圖 0-1 最終架構。API 把任務放進 Redis，Worker 取出後執行 multi-agent 圖，透過 Resilience 層呼叫模擬 LLM，再把結果寫回 Redis。第一到五階段只有「API + 模擬 LLM」兩個服務（API 直接執行 agent 圖）；第六階段起才逐步加入 Nginx、Redis、Worker 和監控。

階段一覽

階段	內容	對應手冊章節	建議時間
0	環境與專案結構	第 8 章（部分）	半天
1	OpenAI 相容的模擬 LLM	第 2、6 章	1 個週末
2	Multi-agent 骨架（同步版）	第 1、2 章	1 個週末
3	Baseline 壓測	第 4、5 章	2–3 個晚上
4	Async 改造與平行 agent	第 1、3 章	1 個週末
5	容錯與故障注入	第 7 章	1 個週末
6	容器化與水平擴展	第 8、9 章	1 個週末
7	任務佇列與冪等性	第 10 章	1–2 個週末
8	快取	第 11 章	2–3 個晚上
9	監控儀表板	第 12 章	1 個週末
10	上雲與雲端壓測	第 13 章	1 個週末
11	成果整理與面試準備	全部	2–3 個晚上

監控（第九階段）也可以提前到第三階段之後做，好處是之後每次壓測都能同時看到系統內部的狀況。初學者照順序做也完全沒問題。

開始前：五條實驗守則

這個專案的價值在於「每個數字都可信、都講得出來源」。以下五條規則從第三階段開始，每次壓測都要遵守。

1. **一次只改一個變數。**例如「sync 改 async」和「加大連線池」要分成兩次實驗，否則不知道是哪個改動造成差異。
2. **先預測，再測量。**每次壓測前，先用 Little's Law 寫下你預期的 RPS 上限和原因。測完比對，差很多就是你最好的學習機會。
3. **暖機 1–2 分鐘，每個階段至少 5 分鐘。**只取穩定期的數據。
4. **同一設定跑 3 次，取中位數。**模擬 LLM 要固定 random seed，讓結果可重現。
5. **每次都寫進實驗紀錄表。**格式見附錄 A。沒寫進表裡的實驗，等於沒做過。

> **兩個常見的陷阱**
> 壓測工具和被測服務搶同一台筆電的 CPU，會讓數字失真。被測服務用 Docker 限制 CPU，並確認 Locust 本身的 CPU 不超過 80%。另外，不要用真的 OpenAI 或 Azure OpenAI 做壓測：費用高、會被限流干擾，而且結果無法重現。真實 LLM 只用在 demo 和最後的少量校正。

建議的專案結構

以下是建議的資料夾結構，可以依自己的習慣調整：

```
multi-agent-loadlab/
├── mock_llm/            模擬 LLM 服務（OpenAI 相容）
├── backend/
│   ├── agents/          四個 agent 與 LangGraph 圖
│   ├── tools/           月付金試算、FAQ 檢索
│   ├── resilience/      timeout、retry、斷路器（第五階段）
│   ├── api/             FastAPI 路由
│   └── worker/          任務佇列 worker（第七階段）
├── data/                假 FAQ、假利率表、壓測用問題池
├── loadtest/            Locust 腳本與 load shape
├── infra/               docker-compose、nginx、prometheus、grafana 設定
├── experiments/
│   ├── log.csv          實驗紀錄表
│   ├── results/         每次 Locust 匯出的 CSV
│   └── charts/          畫好的對比圖
└── README.md
```

0 環境與專案結構

對應手冊：第 8 章（Docker 基礎）

> **這一階段的目標**
> 把開發環境準備好，讓之後每一階段都能專心在學習主題上，不被環境問題卡住。

步驟

1. 建立 GitHub repo 和上一頁的資料夾結構，加入 `.gitignore`（排除虛擬環境、`.env`、壓測結果的大檔案）。
2. 建立 Python 3.12 虛擬環境，安裝 FastAPI、Uvicorn、httpx、requests、LangGraph、langchain-openai、Locust、pandas、matplotlib。
3. 安裝 Docker Desktop，確認能跑起一個簡單的容器。
4. 建立 `.env.example`，列出之後會用到的設定，例如模擬 LLM 的網址、延遲範圍、限流數值。真正的 `.env` 不進版控。
5. 建立 `experiments/log.csv`，欄位照附錄 A。
6. 寫好假資料：約 30 條 FAQ、一張假利率表、一份 50 個問題的「壓測問題池」。問題池要涵蓋只需要知識、只需要試算、兩者都需要的三種類型，比例大約 4：3：3。

完成檢查

- ☐ repo 已建立，結構清楚，第一個 commit 已推上 GitHub
- ☐ 虛擬環境能匯入所有套件，Docker 能正常執行
- ☐ 假資料和問題池已備妥

1 OpenAI 相容的模擬 LLM

對應手冊：第 2 章（HTTP 與 FastAPI）、第 6 章（LLM API 與限流）

> **這一階段的目標**
> 做出一個「假的 Azure OpenAI」：格式一樣、延遲合理、會限流、會偶爾出錯，而且所有行為都能用環境變數控制。之後所有實驗的可信度都建立在它身上。

步驟

1. **實作 `POST /v1/chat/completions`**。回傳格式照手冊 6.1：包含 `id`、`choices[0].message`、`finish_reason`、`usage`。endpoint 用 `async def`，延遲用 `asyncio.sleep`。如果誤用 `time.sleep`，模擬服務自己就會變成瓶頸，之後所有實驗都量錯東西。
2. **讓回覆內容「看起來合理」**。multi-agent 需要 supervisor 回傳路由決定，所以模擬 LLM 要依角色回不同內容。做法是在每個 agent 的 system prompt 開頭放一個標記，例如 `[ROLE:supervisor]`，模擬服務看到標記就回對應的罐頭回覆。Supervisor 要回一段 JSON，說明要找哪些專家；Reviewer 回「通過」或修正後的文字。
3. **設計延遲模型**。延遲 = 固定開銷 + completion tokens × 每 token 時間，再加一點隨機。這樣回答越長越慢，比單純的 1–3 秒隨機更接近真實。
4. **實作雙重 token bucket 限流**。一個管 TPM，一個管 RPM，參考手冊 6.3。token 數用「prompt 長度估算 + max_tokens」計算，和 Azure 的算法一致。
5. **被限流時回 429，並附上 `Retry-After` header**。值用 token bucket 算出「還要等幾秒才夠」。
6. **加入故障注入**。用環境變數控制：隨機回 500 的機率、隨機「卡住很久」的機率、整個服務暫停回應的開關。第五階段會大量使用。
7. **所有參數都用環境變數控制，並固定 random seed**。
8. **只開 1 個 worker**。多個 worker 會各有一個 token bucket，限流就不準了。

驗證方法

- 用 `curl` 或 Swagger UI 送一個請求，確認回傳格式正確。
- 寫一個小腳本，用 LangChain 的 `ChatOpenAI` 把 `base_url` 指向模擬服務，確認能正常拿到回答。這證明之後 agent 程式碼不用改，就能在真假 LLM 間切換。
- 把 RPM 調到很低（例如 10），一次送 30 個請求，確認大約 20 個收到 429，而且都有 `Retry-After`。

完成檢查

☐ LangChain 能直接把模擬服務當成 OpenAI 使用

☐ 不同角色標記會得到不同的回覆

☐ 限流、429、Retry-After、故障注入都能用環境變數開關

☐ 同樣的 seed 跑兩次，結果一致

2 Multi-agent 骨架（同步版）

對應手冊：第 1 章（併發基礎）、第 2 章（FastAPI）

> **這一階段的目標**
> 做出功能正確的多代理人後端，而且**刻意先用同步寫法**。它是第三階段的 baseline，也是第四階段對比的起點。

步驟

1. **先在紙上畫出 agent 流程圖。**Supervisor → 一或兩個專家 → Reviewer → 回覆。標出哪些步驟有先後依賴、哪些可以同時進行。這張圖之後會放進 README。
2. **定義共享狀態（state）。**至少包含：對話紀錄、supervisor 的路由決定、各專家的結果、最終回答、每一步花的時間和 token 數。記錄每一步的耗時，之後找瓶頸會很有用。
3. **寫兩個工具。**月付金試算（純 Python 公式）和 FAQ 檢索（先用關鍵字比對就好，不需要向量資料庫）。
4. **用 LangGraph 組出圖。**每個 agent 是一個節點，supervisor 的決定透過條件邊導到對應的專家。這一版先讓兩個專家依序執行。
5. **LLM 設定集中在一處。**模型名稱、`base_url`、timeout 都從環境變數讀，切換真假 LLM 只需改設定。
6. **寫 FastAPI 的 `POST /chat`。**這一版用 `def` endpoint，呼叫 `graph.invoke`（同步）。回傳最終回答，以及這次請求的 LLM 呼叫次數和總耗時。
7. **加上 `GET /health`。**之後 Docker、Nginx 和雲端平台都會用到。
8. **對話記憶先放在記憶體**（例如 LangGraph 的 in-memory checkpointer）。第六階段水平擴展時，你會親眼看到它出問題，再改到 Redis。

驗證方法

- 從問題池三種類型各挑 3 題手動測試，確認 supervisor 路由正確。
- 確認每個請求的 LLM 呼叫次數在 3–4 次。
- 選做：把 `base_url` 換成真的 LLM 跑 5 題，確認真實模型下也能正常運作，然後切回模擬服務。

完成檢查

☐ 三種類型的問題都能得到合理的回答

☐ 回應中包含 LLM 呼叫次數和耗時

☐ agent 流程圖已畫好並放進 README

3 Baseline 壓測

對應手冊：第 4 章（效能指標、Little's Law）、第 5 章（壓測方法論與 Locust）

> **這一階段的目標**
> 量出同步版的容量上限，並學會「先預測、再驗證」的實驗流程。這一階段的圖是之後所有改進的比較基準。

步驟

1. **先寫下預測。**用手冊 2.5 和 4.4 的方法：FastAPI threadpool 約 40 個 thread，每個請求約 W 秒（用第二階段量到的平均耗時），預測 RPS 上限 = 40 ÷ W。寫進實驗紀錄的「預測」欄。
2. **寫 Locust 腳本。**每個虛擬使用者從問題池隨機抽題送到 `/chat`。用 `catch_response` 把 429、5xx、timeout 分開標記成失敗。
3. **寫階梯式 load shape。**10 → 25 → 50 → 100 → 150 → 200 個使用者，每階 5 分鐘。
4. **限制被測服務的資源。**後端和模擬 LLM 各限 1 個 CPU（可以先用 Docker 跑，或者這階段先在本機直接跑，第六階段再容器化）。模擬 LLM 的限流先設得寬鬆，讓瓶頸只出現在後端。
5. **用 headless 模式執行，匯出 CSV。**同一設定跑 3 次。
6. **畫兩張圖。**X 軸都是使用者數；一張的 Y 軸是 RPS，一張是 p50、p95、p99。
7. **找出「拐點」。**RPS 不再上升、p95 開始急升的那一點，就是系統容量。和你的預測比較。

要回答的問題

- 實測上限和預測差多少？如果差很多，原因可能是什麼（平均耗時估錯、CPU 被吃滿、Locust 本身太忙）？
- 超過拐點之後，延遲為什麼會像曲棍球棒一樣暴增？用排隊的概念解釋。
- 這時候 CPU 使用率是多少？如果很低，代表瓶頸是什麼？

完成檢查

☐ 實驗紀錄表至少有 3 列 baseline 數據

☐ RPS 圖和延遲圖已存進 `experiments/charts/`

☐ 能用一段話解釋「為什麼同步版大約在這個 RPS 撞牆」

4 Async 改造與平行 agent

對應手冊：第 1 章（coroutine、gather、Semaphore）、第 3 章（httpx 與連線池）

> **這一階段的目標**
> 把整條呼叫鏈改成 async，並讓互不依賴的專家 agent 同時執行。這會產生你履歷上第一個、也是最有說服力的數字。

步驟

以下每一步做完都要重測，各自記成一列實驗，才分得出每個改動的貢獻。

1. **4a：endpoint 與 graph 改成 async。** `/chat` 改成 `async def`，`graph.invoke` 改成 `await graph.ainvoke`；每個 agent 節點也改用 `ainvoke`。工具如果是同步的 I/O，用 `asyncio.to_thread` 包起來。
2. **4b：檢查有沒有漏網的阻塞呼叫。**用手冊 1.2.3 的心跳實驗概念：同時送 20 個請求，總時間應該接近單一請求的時間。如果接近 20 倍，代表某處還在阻塞 event loop。
3. **4c：調整連線池。**先保持預設連線池重測一次，再依 Little's Law 計算需要的連線數並調整，再測一次。觀察有沒有出現 `PoolTimeout`。
4. **4d：讓專家 agent 平行執行。**問題同時需要知識和試算時，兩個專家用 LangGraph 的平行分支（或 `gather`）同時跑。只對「兩者都需要」的那類問題有效，所以要分開統計這類問題的延遲。
5. **4e：限制同時打 LLM 的數量。**用 Semaphore 設上限，上限值依模擬 LLM 的限流反推。比較加了上限前後的 429 數量。

要畫的圖

- **sync vs async 主圖：**兩條線畫在同一張 RPS 圖和同一張 p95 圖上。這是整個專案最重要的一張圖。
- **平行 agent 前後：**只看「知識＋試算」那類問題的 p50 和 p95。

完成檢查

☐ 20 個同時請求的總時間接近單一請求

☐ 4a 到 4e 每一步都有獨立的實驗紀錄

☐ 能說出「async 版的新瓶頸是什麼」（通常是連線池或模擬 LLM 的限流）

> **常見坑**
> LangChain 和 OpenAI SDK 預設就會自動重試。這一階段先把 `max_retries` 設成 0，讓錯誤原樣呈現；第五階段再自己實作重試，才看得出效果。

5 容錯與故障注入

對應手冊：第 7 章（timeout、retry、circuit breaker、idempotency）

> **這一階段的目標**
> 讓系統在「下游會出錯」的情況下仍然穩定，並用故障注入實驗證明每一項機制真的有效。

步驟（依序加入，每加一項就重測）

1. **5a：timeout 預算。**每個使用者請求設總時限（例如 30 秒），每次 LLM 呼叫用「剩餘時間」當 timeout。超時要回傳清楚的錯誤，而不是讓請求一直掛著。
2. **5b：retry with exponential backoff + full jitter。**只重試 429、502、503、504 和連線錯誤；有 `Retry-After` 就照它等；設最大重試次數；並確認重試不會超過 5a 的總時限。
3. **5c：circuit breaker。**連續失敗達門檻就跳開，直接快速失敗；過一段時間進入 half-open 試探。把 breaker 的狀態變化記到 log。
4. **5d：降級回應（fallback）。**某個專家 agent 失敗時，不要讓整個請求失敗，而是回傳「部分回答＋說明哪部分暫時無法提供」。這是 multi-agent 系統特有的容錯設計，面試很加分。

故障注入實驗

實驗	怎麼做	要觀察什麼
限流風暴	把模擬 LLM 的 RPM 調低，用 100 個使用者壓測	有無 jitter 時，429 數量和成功率的差異
偶發錯誤	模擬 LLM 隨機 5% 回 500	retry 前後，使用者看到的錯誤率
下游中斷	壓測進行到一半，讓模擬 LLM 停止回應 30 秒	有無 breaker 時的 p99、恢復速度、資源是否被耗盡
慢回應	隨機 5% 的呼叫延遲到 20 秒	timeout 預算能否保住 p99
突發流量（spike）	10 個使用者瞬間跳到 200 個	系統會不會雪崩，多久回到穩定

要回答的問題

- 為什麼只有 backoff、沒有 jitter 時，429 會成群出現？
- retry 讓錯誤率下降，但對 p99 有什麼影響？這個取捨值得嗎？
- 如果前端、後端、SDK 三層都重試 3 次，最壞情況會發生什麼？

完成檢查

☐ 四項機制都已加入，每項都有獨立的實驗紀錄

☐ 「下游中斷」實驗有一張時間序列圖，清楚看到 breaker 跳開與恢復

☐ 某個專家失敗時，使用者仍能拿到部分回答

6 容器化與水平擴展

對應手冊：第 8 章（Docker Compose）、第 9 章（Nginx 與 stateless）

> **這一階段的目標**
> 把系統拆成多個容器，用 Nginx 分流到 3 個後端，並親身體驗「為什麼水平擴展要求 stateless」和「為什麼加機器不一定有用」。

步驟

1. **為模擬 LLM 和後端各寫一個 Dockerfile**。注意 layer 順序（先裝套件再複製程式碼）和 `--host 0.0.0.0`。
2. **寫 docker-compose**。每個服務限制 1 個 CPU 和固定記憶體，設定透過環境變數傳入，後端用服務名稱連到模擬 LLM。
3. **先在 Compose 裡重跑一次第四階段的最佳設定**。確認容器化後的數字和之前接近，作為這一階段的新 baseline。
4. **加入 Nginx 和 3 個後端**。upstream 用 least_conn，設定 keepalive 和 `proxy_read_timeout`（要大於 agent 最長執行時間）。
5. **觀察 stateless 問題**。用同一個 session 連續問兩個有上下文關係的問題，會發現第二題常常「失憶」，因為它被分到另一台。
6. **把對話記憶搬到 Redis**。加入 Redis 容器，改用 Redis 型的 checkpointer 或自己存對話紀錄。再測一次，確認不再失憶。
7. **同時注意限流的位置**。第四階段的 Semaphore 是「每台」各自的上限，3 台加起來就變成 3 倍。思考要不要改成全域上限（例如用 Redis 實作）。

關鍵實驗：加機器什麼時候有用

設定	預期結果	學到什麼
模擬 LLM 限流寬鬆，1 台 vs 3 台	3 台 RPS 明顯較高	瓶頸在後端時，水平擴展有效
模擬 LLM 限流很緊，1 台 vs 3 台	RPS 幾乎不變，429 變多	瓶頸在下游時，加機器沒用
3 台，round robin vs least_conn	least_conn 的 p99 較低	請求耗時差異大時，演算法的影響

完成檢查

- ☐ 一個指令就能啟動整套系統
- ☐ 多台後端時對話不會失憶
- ☐ 有「寬鬆 vs 很緊」兩組 1 台 vs 3 台的對比圖

7 任務佇列與冪等性

對應手冊：第 10 章（Redis、佇列、at-least-once）、第 7.5 節（idempotency）

> **這一階段的目標**
> 把長時間的 agent 任務改成「送出 → 輪詢」模式，並處理分散式系統最經典的兩個問題：任務重複執行，以及 worker 中途掛掉。

步驟

1. **新增兩個 endpoint。**`POST /tasks` 建立任務並立刻回 202 和 task_id；`GET /tasks/{id}` 回傳狀態（queued、running、done、failed）和結果。原本的同步 `/chat` 保留，作為對照。
2. **先自己用 Redis List 實作一版佇列。**API 把 task_id 推進佇列，worker 取出後執行 multi-agent 圖，結果寫回 Redis 的 hash，並設定 TTL。
3. **加入 idempotency key。**客戶端在 header 帶一個 key，伺服器用 `SET NX` 佔位。同一個 key 重送時，回傳原本的 task_id。
4. **做「worker 掛掉」實驗。**在 worker 執行到一半時強制關掉它，觀察任務會不會永遠停在 running。這就是 at-most-once 的問題。
5. **改成 at-least-once。**用 Redis Streams 的 consumer group，處理完才 ack；沒 ack 的任務由其他 worker 接手。worker 開始執行前先檢查任務是否已完成，避免重複執行。
6. **用 worker 數控制 LLM 併發。**「worker 數 × 每個 worker 的併發數」就是同時打 LLM 的上限，依限流反推。
7. **選做：換成 Celery。**設定 `acks_late` 和 `worker_prefetch_multiplier=1`，和自己的版本比較。
8. **更新 Locust 腳本。**模擬使用者送出任務後，每 1–2 秒輪詢一次直到完成。

要量的指標

- `POST /tasks` 本身的延遲：應該很短，而且幾乎不受負載影響。
- 送出到拿到結果的端到端時間：和同步 `/chat` 比較。
- 突發流量時的佇列長度變化：佇列應該先把尖峰吸收，再慢慢消化。

完成檢查

☐ 重送同一個 idempotency key 不會產生第二個任務

☐ worker 被強制關掉後，任務仍會被完成

☐ 有一張 spike 時佇列長度與處理速度的時間序列圖

8 快取

對應手冊：第 11 章（cache-aside、semantic cache、陷阱）

> **這一階段的目標**
> 在瓶頸是 LLM 配額時，用「不呼叫」來提升容量，並學會快取在正確性和隱私上的風險。

步驟

1. **決定快取的位置。**multi-agent 系統有兩種選擇：快取「整個請求的最終回答」，或快取「單次 LLM 呼叫」。先做單次 LLM 呼叫層級，因為 supervisor 的路由決定重複率最高。
2. **實作 exact cache。**key 要包含模型、完整 messages（含 system prompt）和 temperature，TTL 先設 1 小時。
3. **處理 cache stampede。**熱門 key 過期瞬間，用 `SET NX` 短期鎖只讓一個請求去算。
4. **訂下隱私規則。**含個人資訊的回答（例如某位假客戶的貸款餘額）不快取，或把 user_id 放進 key。寫進 README。
5. **調整壓測問題池。**讓問題有一定重複度，並在實驗紀錄中寫明重複度的假設。
6. **選做：semantic cache。**用 RedisVL，門檻從嚴格開始。準備 10 組「意思相近但答案不同」的問題，量誤命中率。

完成檢查

- ☐ 有命中率、省下的 LLM 呼叫數、命中與未命中時的延遲
- ☐ 有「有快取 vs 無快取」在同一限流下的 RPS 對比圖
- ☐ 隱私規則已寫進 README

9 監控儀表板

對應手冊：第 12 章（Prometheus、Histogram、PromQL、Grafana）

> **這一階段的目標**
> 讓系統「從內部看得見」。壓測時一邊跑 Locust、一邊看 Grafana，就能直接看出瓶頸在哪個環節。

步驟

1. **在每個服務開 `/metrics`**。後端、worker、模擬 LLM 都要。
2. **定義指標**。建議至少包含下表的項目。延遲一律用 Histogram，bucket 要涵蓋 0.5 到 30 秒。
3. **在 Compose 加入 Prometheus 和 Grafana**。設定 Prometheus 抓取所有服務。
4. **做一張 Golden Signals 儀表板**。四格：RPS、錯誤率（429 和 5xx 分開）、p50/p95/p99、飽和度（佇列長度、進行中請求數、連線池使用量）。
5. **再做一張 agent 專屬面板**。每個 agent 的呼叫次數與延遲、circuit breaker 狀態、快取命中率。
6. **重跑一次第五階段的「下游中斷」實驗，這次全程截圖**。這張截圖放進 README 效果很好。

指標	類型	用途
HTTP 請求數（依路由、狀態碼）	Counter	RPS、錯誤率
HTTP 請求延遲	Histogram	對外的 p95、p99
LLM 呼叫延遲（依 agent、結果）	Histogram	哪個 agent 最慢
LLM 錯誤數（依狀態碼）	Counter	429 和 5xx 的比例
重試次數	Counter	retry 是否過度觸發
Circuit breaker 狀態	Gauge	跳開與恢復的時間點
佇列長度、進行中請求數	Gauge	飽和度
快取命中與未命中	Counter	命中率
Token 使用量（依 agent）	Counter	成本與 TPM 用量

完成檢查

- ☐ 沒有使用 user_id、task_id 這類高基數 label
- ☐ 儀表板能清楚顯示一次完整壓測的四個 Golden Signals
- ☐ 有一張故障實驗期間的 Grafana 截圖

10 上雲與雲端壓測

對應手冊：第 13 章（Azure for Students、Container Apps、Azure Load Testing）

> **這一階段的目標**
> 把系統部署到和正式環境相近的雲端，量出可信的絕對數字，並驗證自動擴展。重點是控制花費。

步驟

1. **申請 Azure for Students，並先設預算警示。**所有資源都放在同一個 resource group。
2. **把 image 推到 Azure Container Registry。**
3. **部署到 Azure Container Apps。**後端、worker、模擬 LLM 各一個 container app；Redis 可以先用一個 container app 跑 Redis image 來省錢。每個容器只開 1 個 uvicorn worker。
4. **設定擴展規則。**後端依「同時進行中的 HTTP 請求數」擴展；最大 replica 數用 Little's Law 和模擬 LLM 的限流反推。壓測期間最少 replica 數設 1，避免冷啟動影響數據。
5. **用 Azure Load Testing 執行你的 locustfile。**先確認雲端使用的 Locust 版本和本機相容。
6. **只做兩組實驗，各 10 分鐘。**固定 1 個 replica 和自動擴展 1–5 個 replica。記錄 RPS、p95、錯誤率和 replica 數隨時間的變化。
7. **和本機 Compose 的數字比較，並解釋差異。**例如網路延遲、CPU 規格不同。
8. **實驗結束立刻刪除整個 resource group。**

> **花費提醒**
> Container Apps 的 environment 即使沒有流量也可能產生費用，Container Registry 也會持續收費。每次實驗結束都要刪除資源，下次再用同樣的設定重建。把建立步驟寫成文件或腳本，重建就會很快。

完成檢查

- ☐ 有一張 replica 數隨負載自動增加的截圖
- ☐ 有本機 vs 雲端的數字對照與差異說明
- ☐ resource group 已刪除，帳單在預算內

11 成果整理與面試準備

對應手冊：全部章節

> **這一階段的目標**
> 把做過的事變成別人看得懂、面試講得清楚的成果。

README 應該包含

1. 一句話說明專案：做了什麼、證明了什麼。
2. 架構圖與 agent 流程圖。
3. 如何一鍵啟動和執行壓測。
4. 實驗方法：環境規格、模擬 LLM 的參數、五條實驗守則。
5. 主要結果：5 張對比圖，每張配 2–3 句解讀。
6. 學到的事與限制：例如「數字來自模擬 LLM，真實延遲分佈可能不同」。誠實寫出限制反而更有說服力。

履歷寫法

這是個人專案，所以寫在 Projects 區，不要寫成 Volvo 的成果。從實驗紀錄表挑出最有代表性的數字，填進手冊提供的履歷句型。每一句都要能指出來自紀錄表的哪幾列。

面試要能回答的 10 個問題

1. 為什麼同步版大約在那個 RPS 撞牆？請用 Little's Law 解釋。
2. 改成 async 之後，新的瓶頸是什麼？你怎麼發現的？
3. 為什麼 retry 要加 jitter？你的實驗看到什麼？
4. Retry 和 circuit breaker 的差別？為什麼要一起用？
5. 某個 agent 失敗時，你的系統怎麼處理？
6. 水平擴展時遇到什麼問題？怎麼解決？
7. 為什麼加了 3 台機器，RPS 卻沒有變？
8. 任務可能被執行兩次嗎？你怎麼避免重複的副作用？
9. 快取有什麼風險？你怎麼處理隱私問題？
10. 如果要支援 200 位使用者，你會怎麼估算需要的資源和 LLM 配額？

附錄 A：實驗紀錄表欄位

建議用 CSV 或試算表記錄，每次壓測一列。「預測」和「學到什麼」兩欄最常被省略，但它們才是讓你在面試中講得出故事的關鍵。

欄位	說明與範例
實驗編號	例如 P4-c-run2：第四階段、步驟 c、第 2 次
日期	執行日期
版本／commit	程式碼的 git commit，確保能重現
改了什麼	和上一個實驗相比唯一改變的變數
環境	本機或雲端、CPU 限制、replica 數
模擬 LLM 參數	延遲範圍、TPM、RPM、故障率、seed
負載	使用者數、spawn rate、持續時間、問題池重複度
預測	事前用 Little's Law 推算的 RPS 上限與理由
RPS（成功）	只算成功的請求
p50 / p95 / p99	單位：秒；只算成功的請求
錯誤率	429、5xx、timeout 分開記
飽和度	CPU%、記憶體、連線池使用量、佇列長度
學到什麼	一兩句話：結果和預測差在哪、為什麼

附錄 B：每階段的履歷數字來源

階段	可以產生的數字	比較的兩列實驗
3 → 4	吞吐量提升幾倍、p95 降低多少	sync baseline vs async 最佳設定
4d	平行 agent 讓特定類型問題的延遲降低多少	依序執行 vs 平行執行
5	尖峰負載下錯誤率從多少降到多少	無容錯 vs 完整容錯，同一故障注入
6	水平擴展到幾台、撐到多少 RPS	1 台 vs 3 台，限流寬鬆時
7	送出任務的 p95、突發流量下的錯誤率	同步 /chat vs 佇列模式
8	LLM 呼叫減少多少、成本省多少	無快取 vs 有快取，同一問題池
10	雲端自動擴展下的 RPS 與 p95	固定 1 個 replica vs 自動擴展

提醒：所有數字都來自模擬 LLM 的環境。寫履歷和面試時要主動說明這一點，並解釋為什麼這樣設計（成本可控、結果可重現）。這會被視為成熟的工程判斷，而不是缺點。